

Problem Set 3: Graphs

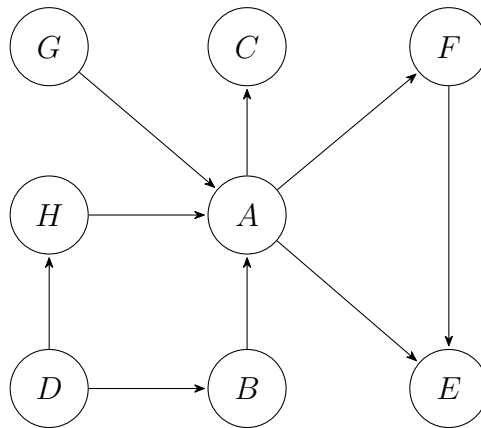
Devin Fromond

Due: September 20th 2023

- Please type your solutions using L^AT_EX or any other software. Handwritten solutions will not be accepted.
- Your algorithms must be in plain English & mathematical expressions, and the pseudo-code is optional. Pseudo-code, without sufficient explanation, will receive no credit.
- Unless otherwise stated, all logarithms are to base two.
- If we ask for a specific running time, a correct solution achieving it will receive full credit even if a faster solution exists.

Problem 1: Topological Sort

(20 points)



Using the above graph G , answer the following questions.

- (a) Processing nodes in alphabetical order, perform a Depth-First Search on G . Break ties alphabetically. Write the pre and post numbers of all vertices (starting from 1).

Solution: We will answer this question with an ordered pair in the form $(pre, post)$ number.

A) (1,8)

B)

C) (2,3)

D)

E) (4,5)

F) (6,7)

G)

H)

It is important to note B, D, G, and H will not be visited since *explore()* will only find connected nodes.

- (b) What are the strongly connected components of G ?

Solution: Since G does not contain a cycle, there are no strongly components of G of more than two vertices. As such, we arrive at our trivial answer of:

$$\{A\}, \{B\}, \{C\}, \{D\}, \{E\}, \{F\}, \{G\}, \{H\}$$

(c) Give a topological ordering of G if one exists.

Solution: $[G, D, H, B, A, F, E, C]$

Problem 2: Multiple Monkeys

(20 points) Given a set of monkeys $M \subseteq V$ and a set of bananas $B \subseteq V$ as vertices in an unweighted, undirected graph G . Find the shortest path in the graph (with respect to the number of edges) from any monkey to any banana in linear time. Formally, design an algorithm to find the shortest path in G such that it starts at some monkey $m \in M$ and finishes at any banana $b \in B$. Provide an explanation on correctness and runtime.

$$\min_{m \in M, b \in B} dist(m, b)$$

Note: not all vertices are labelled as monkeys or bananas!

Solution:

- *Algorithm:* It is unclear in the instructions and on EdStem posts if the input to the question is a graph G AND nodes (which can be monkey, banana, or neither) AND edges, or if the input is a graph G WITH nodes (...) AND edges (nodes/edges are not input). If G is the only input, we will first traverse the graph to find all Monkey nodes and store them in a set. The rest of the problem is assuming we have a set of Monkeys somewhere.

We will utilize a multi-BFS or simultaneous BFS algorithm that will be performed on each Monkey node at once. Each monkey will be marked as Visited, or have $dist = 0$, depending on our backing data structure.

The BFS algorithm will work identical in terms of a traditional BFS with regard to traversal. After a vertex is explored, it will check if the vertex is a banana. If it is a banana, the path will be returned since we know the found banana is the closest banana to the monkey - due to the properties of BFS. If the vertex is not a banana, it will continue. This process will continue until one monkey finds a banana, which we discussed previously, or until it has no further vertices to explore, which will return null.

- *Correctness:* Due to the traversal property of BFS that we utilize, we guarantee that in the worst-case, every vertex and every edge will be explored. If a banana is present, the algorithm is guaranteed to return the shortest path since BFS, by design, explores the closest vertices first. Once a banana is explored, the path to the banana will be returned and the algorithm will end.
- *Time Complexity Analysis:* We find the runtime of the algorithm to be $O(|V| + |E|)$ since every vertex and edge will be explored at most one time. This is the worst-case runtime of the algorithm since the average case will terminate once a banana is found. In the case where we must iterate over the nodes to find the monkeys, as explained in the italicised section of my answer, this process is $O(|V| + |E|)$, which would still make the runtime $O(|V| + |E|)$ since $(O(2|V| + 2|E|)) = O(|V| + |E|)$.

Problem 3: Truth Finder

(20 points)

In propositional logic, a 2-clause is an OR ($\vee$) of exactly two propositional variables or negated propositional variables. Some examples may include: $(p \vee q)$, $(p \vee \neg q)$, $(\neg p \vee q)$, $(\neg p \vee \neg q)$.

Suppose you are given a proposition which is the AND ($\wedge$) of many 2-clauses, such as

$$(x \vee y) \wedge (\neg a \vee \neg x) \wedge \dots \wedge (\neg z \vee w)$$

Design an algorithm which takes as input a proposition of this form. Your algorithm should return a TRUE if there is an assignment to the propositional variables to make the proposition true and FALSE otherwise. You may suppose the input has n propositional variables and your runtime should be polynomial in n . Provide an explanation of runtime and correctness.

Example One:

Input: $(x \vee \neg y) \wedge (y \vee \neg z) \wedge (\neg z \vee \neg x)$

Output: TRUE

Explanation: The assignment $x = T, y = T, z = F$ makes the proposition true.

Example Two:

Input: $(\neg x \vee \neg y) \wedge (x \vee \neg y) \wedge (\neg x \vee y) \wedge (x \vee y)$

Explanation: There is no assignment of the propositional variables that makes the proposition true.

- (Hint 1): $\neg p \vee q \equiv p \implies q$
- (Hint 2): Such a formula has no solution if and only if there is any propositional variable p such that $p \iff \neg p$.

Solution:

- *Algorithm:* Well, this is an graph-related homework. Let's turn it into a graph... We wish to use an implication graph to express the relationships.

We treat each literal, x and $\hat{x}$, as a vertex. Each clause, $(x \vee y)$, can be written as $\hat{x} \implies y$ and $\hat{y} \implies x$. Therefore, for each clause, we must add two directed edges to the graph, $\mathbf{G}$.

Since we are a theoretical course, we will not discuss implementation, so we find it sufficient to say we will implement based off of the three rules above.

We wish to utilize a modified DFS, as discussed in lecture, to approach the problem. We utilize two sets: one called **visited** to store the visited nodes and the other called **recVisited** to store the nodes visited in the recursion calls. These two sets set the current vertex to TRUE.

For every edge in $\mathbf{G}$, we will check if the current vertex in **recVisited** is TRUE; if it is, we know a cycle is detected. Therefore, we must ensure the opposite vertex (ex. if $v = x$, then $\hat{v} = \hat{x}$) is not in the cycle. We can do this by checking the **recVisited** set and print **TRUE** if the opposite vertex is in the set. We will also check if the

current vertex is in **visited**; if it is not, we call **explore()** (which is this method) on the graph using **if explore(G,u)**, where **u** is the adjacent nodes. If this statement is **True**, we will return **TRUE**.

Outside of the for loop, we wish to set the current vertex to **FALSE** in **recVisited** and return **FALSE**.

- *Correctness:* There are two components of the algorithm: (1) graphify the problem and (2) solve the problem. We find the first part to be correct due to Hint 1, which confirms the second rule we use to form the graph. For the second part of the graph, we use modified DFS, which we know works, but the question here is about whether a cycle constitutes the proposition as invalid. As such, We use Hint 2, which describes there should not be a path from p to $\hat{p}$ and a path from $\hat{p}$ to p . In other words, there should not be a cycle containing p and $\hat{p}$, which we see our algorithm checks for.
- *Time Complexity Analysis:* There are three non-time-negligible parts of this algorithm: (1) creating the implication graph, (2) checking for a cycle using modified DFS, and (3) checking if $\hat{v}$ is in **recVisited**. For the first part, we know there are n literals. We assume there are no duplicate clauses. Since each clause involves at most two literals, we know the graph **G** takes $O(m)$ time, where m is the number of clauses, which is at most n . Therefore, we arrive at the worst-case time complexity of $O(n)$. For the second part, DFS takes $O(m+n)$, with some constant time added in there (and removed for simplicity) due to two edges for each clause. Since $m \leq n$, we again have $O(n)$. Finally, iterating through a set of the vertices takes $O(n)$ time. Therefore, we arrive at our final time complexity of $O(n)$.

Problem 4: Navigating Atlanta

(20 points)

The City of Atlanta, Georgia includes the most extensive pedestrian skyway system in the world. It contains over 15 kilometers of heated pedestrian walkways to connect local landmarks and shield pedestrians from the city's harsh winter climate. Unfortunately, residents have started complaining that this system is difficult to navigate. You are a software engineer working on Hooli Maps, a navigational application, and have decided to tackle this problem by calculating directions.

Your Data Analysis team gives you a list of lists of city blocks (represented as integers from 1 to n) that each particular skyway covers. For example, your data analysis team may offer $[[1, 2, 3], [2, 4, 5]]$, indicating that skyway 1 covers blocks 1, 2, and 3, and skyway 2 covers blocks 2, 4, and 5. The walk between any two city blocks takes far too long and would expose pedestrians to hypothermia, but if multiple skyways both cover the same city block, then pedestrians can exit the system at this block and transfer to any of the other skyways covering it. The pedestrian will have to be outside for the duration of the transfer.

Because your mapping software works outside these skyways, you have an $\mathcal{O}(1)$ blackbox that tells you how much time it takes to transfer, given the indices of the two skyways in the transfer, and the integer corresponding to the block at which to complete that transfer. Note that the result will not change if the ordering of the skyways are changed.

Given two integer city blocks b_0 , b_1 , and a list of s skyways, each represented as an array of city block integers from 1 to n , design an efficient algorithm to calculate the minimum amount of time a pedestrian has to be outside to navigate between the two blocks. Provide an explanation on runtime and correctness.

Solution:

- *Algorithm:* We assume the graph of skyways between blocks is not dense. Therefore, we wish to use Dijkstra's Algorithm with a Binary Heap for the Priority Queue, as discussed in class.

We wish to represent the problem using a graph, where skyways serve as nodes. An edge will connect two skyways if they share at least one city block. The weight of this edge, which corresponds to the time it takes to transfer from one skyway to another, will be determined using the provided blackbox algorithm.

To find the shortest path between two blocks, b_0 and b_1 , we will utilize a Priority Queue based on a Binary Heap. The Priority Queue stores the skyways and expands the skyway with the current minimum transfer time first. This property is identical to the core functionality of Dijkstra's algorithm.

We begin by initializing all edges to $+\text{inf}$, except for b_0 , which we initialize to 0. We then use the Binary Heap to expand the skyway with the least current transfer time. For each edge, we will use the blackbox function and update the Binary Heap accordingly. If the new transfer time is smaller than the current known time for that

skyway, we update the time and push the skyway back into the Binary Heap. We continue this process until we reach b_1 , where we must check for b_1 at every node. After b_1 is reached, we return the accumulated transfer time found in the Priority Queue.

- *Correctness:* The correctness of the algorithm is guaranteed by Dijkstra's Algorithm which always expands the node with the smallest current transfer time first, ensuring once a transfer time is finalized, it represents the minimum time to reach that skyway. Additionally, the correctness is reinforced by the way in which we construct the Binary Heap: having two nodes connect to one another if and only if they share a minimum of one city block. This ensures, by use of a skyway, an Atlantaian can reach any block from one city block to another if it has an edge, with the transfer time (or edge) being calculated by the blackbox algorithm.
- *Time Complexity Analysis:* Since we assume the graph of skyways between city blocks is not dense, it is most efficient to use a Binary Heap implementation for Priority Queue, which is $O((V + E)\log(V))$. To create the Binary Heap, we will assume a basic heapify implementation which takes $O(V)$. To check if a the target city block b_1 , we must iterate through the city blocks the particular skyway connects to. We define a new variable m which represents the maximum number of city blocks that is part of a skyway. Therefore, the worst case for checking for b_2 will take $O(V \cdot m)$. Therefore, we arrive at our final time complexity of $O((V \cdot m) + (V + E)\log(V))$.

Problem 5: Badminton

(20 points)

A badminton club in Ladhaland has students from several universities. You are given a list of pairs of player IDs, where each pair is made up of players from the same university. However, there is no information provided on what university each pair is from. What you do know is that each university has exactly one designated badminton captain and that their player ID is in at least one of the pairs. You don't know what player IDs are captains.

Design an algorithm to find the number of **non captain** players in the badminton club. You can assume that every player appears in the provided list of pairs of player IDs and that there are at least 2 players from each university in the city that are a part of the badminton club. Your algorithm should run in worst case $O(E)$ complexity, where E is the number of given pairs. Provide an explanation of correctness and runtime.

Solution:

- *Algorithm:* We wish to use a modified Union-Find algorithm for the problem. We begin by initializing the Disjoint Set Data Structure in which each pair is already in a set. We associate each set with a **size** variable which represents the number of IDs in the set. At this step, we will have E sets, each with a size of 2. Next, we will combine sets according to the pairs of IDs. This will group players into the same university, updating the count each time a union is made. After the Union/Find functions occur, we will be left with groups of universities with a size count. Since each university has one captain, we know the number of non-captains is **size-1**. We then sum up all non-captain players from all unique universities and return the value.
- *Correctness:* We guarantee correctness using the Union/Find algorithm which will merge sets based off of a mutual connection. By this, we guarantee at the end of the algorithm each set represents a unique university through which a simple calculation returns the number of non-captains for all universities.
- *Time Complexity Analysis:* As discussed in lecture, the runtime for Union/Find is $O(V)$. Since we start with E sets, we observe a runtime of $O(E)$. When summing the total number of non-captains, we arrive at a worst-case scenario that every pair is a distinct university. Therefore, performing calculations takes $O(E)$ time. Therefore, together, the algorithm has a $O(2E) = O(E)$ runtime.